

Full-Stack Intern Assignment

Movie Discovery App

1. Problem Statement

Build a movie discovery application that allows users to explore a large collection of movies and find titles they are interested in.

Use a suitable public third-party movie API as the source of movie information.

The application should feel like a real movie discovery product rather than a simple API demonstration. We are interested not only in the final UI, but also in how you approach product requirements, structure a full-stack application, make technical decisions, and handle real-world scenarios.

2. User Experience

When users open the application, they should be able to browse movies and discover something interesting without having to search for a specific title.

Users should be able to:

- Search for movies.
- Explore movies using relevant categories or attributes.
- Change how results are ordered.
- Continue exploring when there are many matching results.
- Open a movie to view more details.
- Add movies to a persistent wishlist and access them again later, including after closing and reopening the application.
- Navigate between browsing, movie details, and their wishlist without losing their context.

The experience should provide appropriate feedback while information is loading, when there are no results, and when something goes wrong.

The application should remain usable as the amount of content increases.

3. External Movie Data

Movie information will come from a third-party service.

The client should communicate with **your Node.js backend**, rather than directly with the external movie API.

Your backend should act as an abstraction layer between the application and the external service. The data provided by the external API may not be in the format your application needs, so decide how it should be represented and exposed to the client.

Consider the behaviour of the application when:

- The same information is requested repeatedly.
- The user performs multiple searches or changes filters quickly.
- The external service is slow or temporarily unavailable.
- The external service returns incomplete or unexpected information.
- The external service has limitations on how frequently it can be accessed.

The application should handle these situations gracefully.

4. Technology

Choose one of the following:

- **React + Node.js**
- **React Native + Node.js**

You may choose the database and supporting libraries based on your approach.

The application should adapt naturally to different screen sizes and devices.

Consider scenarios such as:

- Different screen widths.
 - Movie posters with different dimensions.
 - Long movie titles.
 - Large result sets.
 - Slow network conditions.
 - Empty results.
 - Failed requests.
 - Users quickly changing their search or filters.
-

5. Technical Decisions

Some implementation details are intentionally left open.

You should determine:

- How the frontend should be structured.
- How your Node.js API should be designed.
- How movie data should flow between the external service, backend, database, and client.
- How wishlist data should be persisted.
- What data should be stored by your application versus retrieved from the external service.
- How to efficiently handle a large number of results.
- How to avoid unnecessary requests.
- How to handle failures and inconsistent external data.
- How to keep the application maintainable as it grows.

There is no single correct solution. We are interested in the reasoning behind your decisions and how well they fit the problem.

6. Use of AI

AI-assisted development tools are **allowed and encouraged as supporting tools**.

You may use tools such as ChatGPT, Claude, GitHub Copilot, Cursor, Gemini, or similar tools to help with research, documentation, debugging, boilerplate, code review, or exploring possible solutions.

However, you are expected to understand the code you submit and take ownership of the final implementation.

Using AI extensively will not automatically reduce your evaluation. Your evaluation will depend on your ability to understand, explain, debug, modify, and extend your implementation.

For example, it is completely acceptable to use AI to:

- Understand an unfamiliar API or library.
- Generate initial boilerplate.
- Investigate errors.
- Explore alternative approaches.
- Review or improve code.
- Handle repetitive implementation work.
- Understand technical documentation.

Simply submitting an AI-generated solution without understanding it may result in a lower evaluation.

During the review, you may be asked to:

- Explain important parts of your implementation.
- Explain your architectural and technical decisions.
- Trace how data moves through your application.
- Identify potential problems or limitations.
- Debug an issue.
- Modify an existing feature.
- Implement an additional requirement.

You should therefore be able to work with and explain your implementation regardless of the amount of AI assistance used.

7. AI Transparency

If AI played a significant role in your implementation, briefly mention how you used it in your README.

For example:

"Used AI to understand the third-party API documentation, generate initial API boilerplate, and troubleshoot request handling. The API architecture, database structure, and application behaviour were based on my own decisions."

There is no need to hide your use of AI. We are interested in your ability to use modern development tools effectively while maintaining a strong understanding of the resulting solution.

8. Deliverables

Submit:

1. A working application.
2. Frontend source code.
3. Node.js backend.
4. Database/schema, where applicable.
5. Git repository.
6. README containing:
 - Setup instructions.
 - Approach taken.

- Important technical decisions.
 - Assumptions made.
 - Known limitations.
 - AI tools used and how they were used, if applicable.
 - What you would improve with additional time.
-

9. Evaluation & Review

The application will be evaluated on both the final result and the approach taken.

We will consider:

- Product thinking and UX.
- Frontend implementation.
- Backend architecture.
- Third-party API integration.
- Data modelling and persistence.
- Handling of large result sets.
- Performance and efficiency.
- Error and edge-case handling.
- Responsive design.
- Code quality and maintainability.
- Technical decision-making.
- Ability to explain, debug, and extend the implementation.

A short follow-up discussion or live coding session may be conducted after submission. You may be given an additional scenario or requirement related to your implementation.

The purpose of the review is not to test memorization of syntax. It is to understand how you **think, build, troubleshoot, make decisions, and work with modern development tools.**